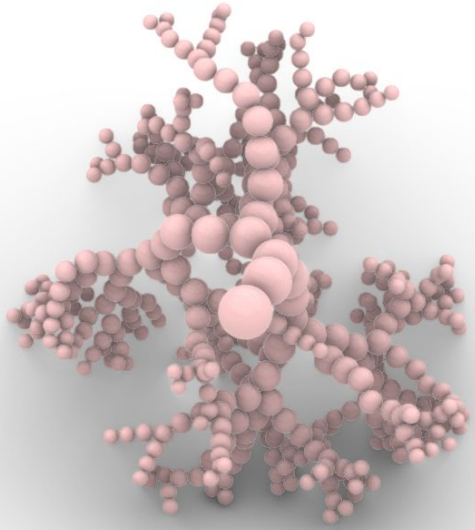
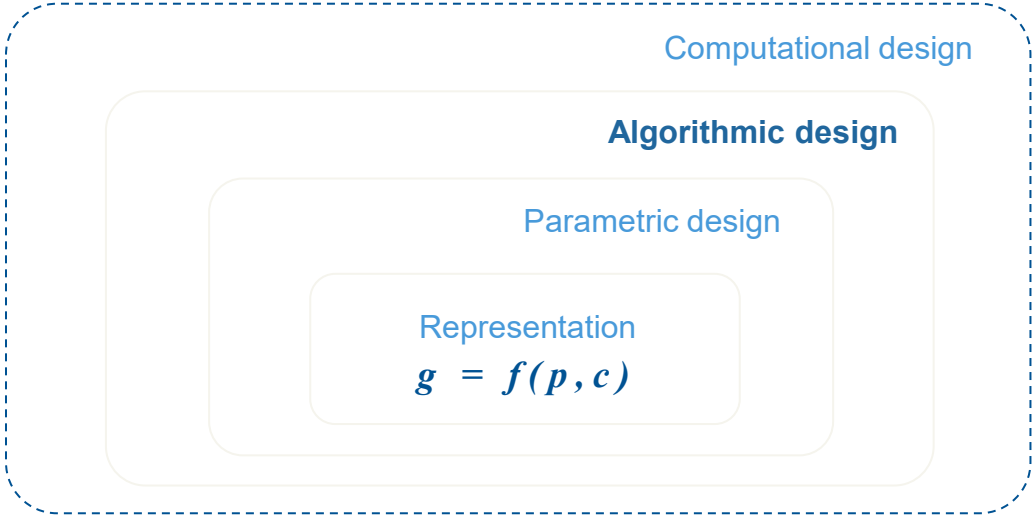
Algorithmic Design

Lecture 3.0 | 9 November 2026



Computational Design < > Algorithmic Design



Representation — the designer edits **g** directly

= Drawing and modelling the shape itself

Parametric — the designer sets **p, c** of a given f

= Parameters, Constraints and Relations in a fixed model

Algorithmic — the designer writes **f**

= Rules, procedures, simulations

Sources: Kotnik, T.: Digital Architectural Design as Exploration of Computable Functions, 2010. Caetano, I. et al.: Computational Design in Architecture, 2020.

What is algorithmic design?

An algorithm is **any well-defined computational procedure** that takes a set of values as input and produces a set of values as output in a finite amount of time.

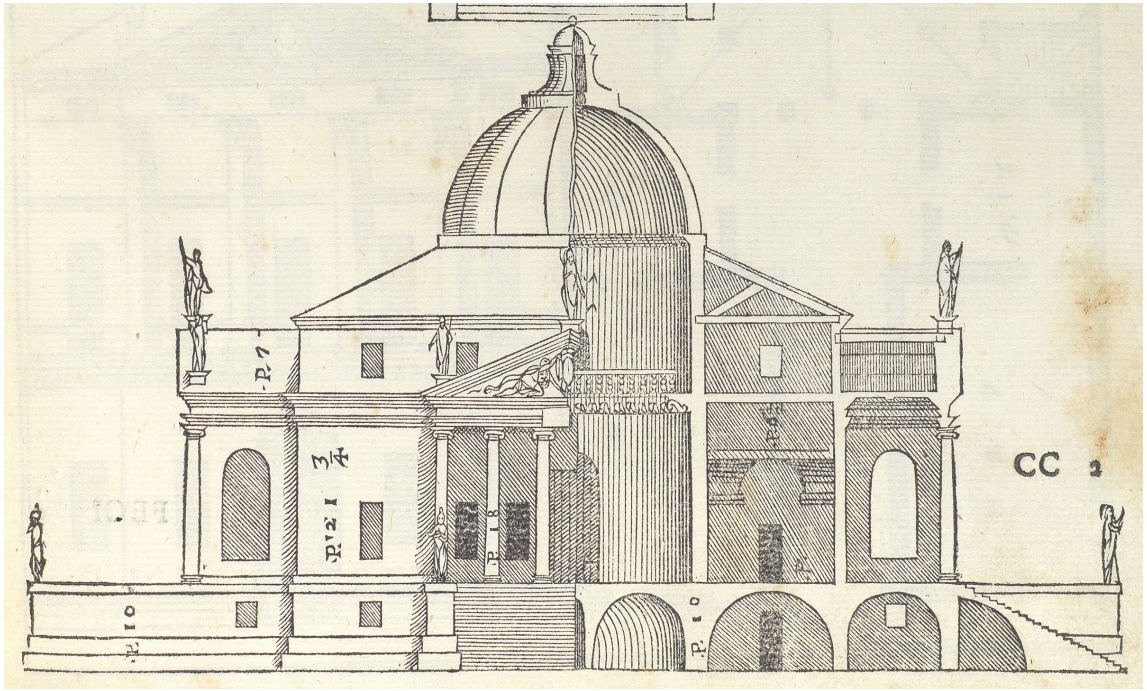
Algorithmic design refers to the **development of rules and computational procedures,** such as a sequence of algebraic, analytic, and geometric operations, for the manipulation of data and its translation into architectural geometry.

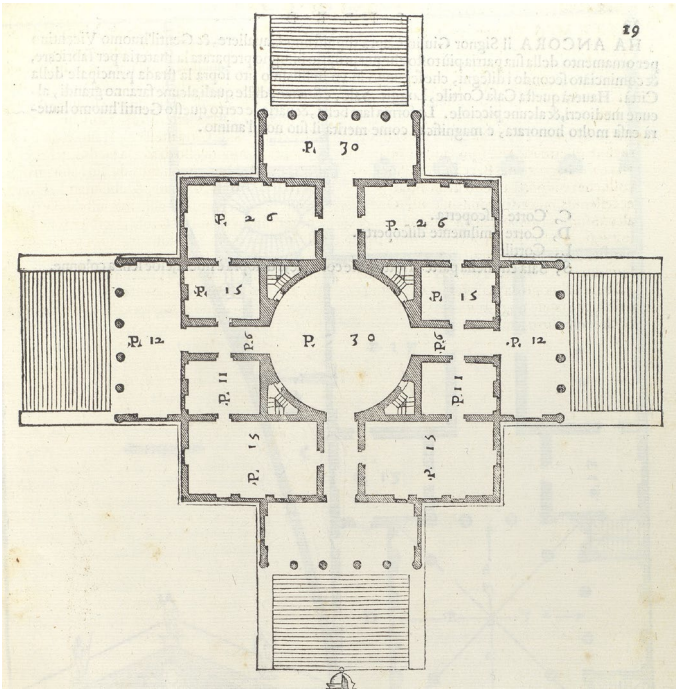


Sources: Cormen, T.H. et al.: Introduction to Algorithms (4th ed.), 2022. Kotnik, T.: Digital Architectural Design as Exploration of Computable Functions, 2010

Architecture and Algorithmic Design

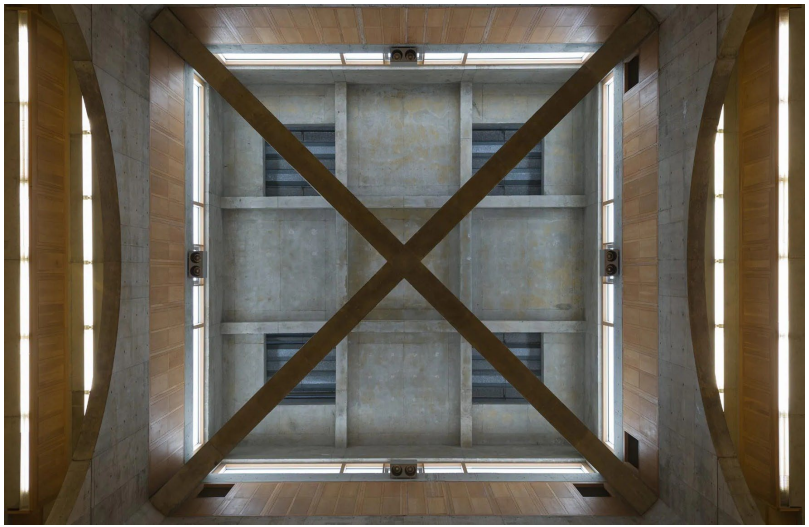
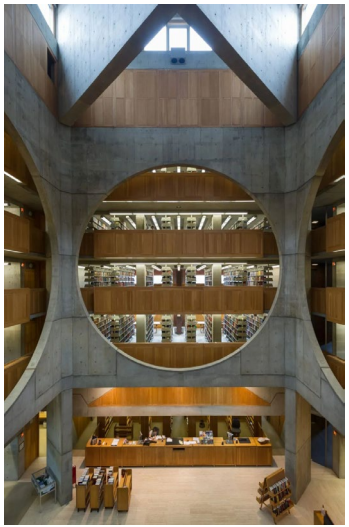
Architecture as a system of proportions





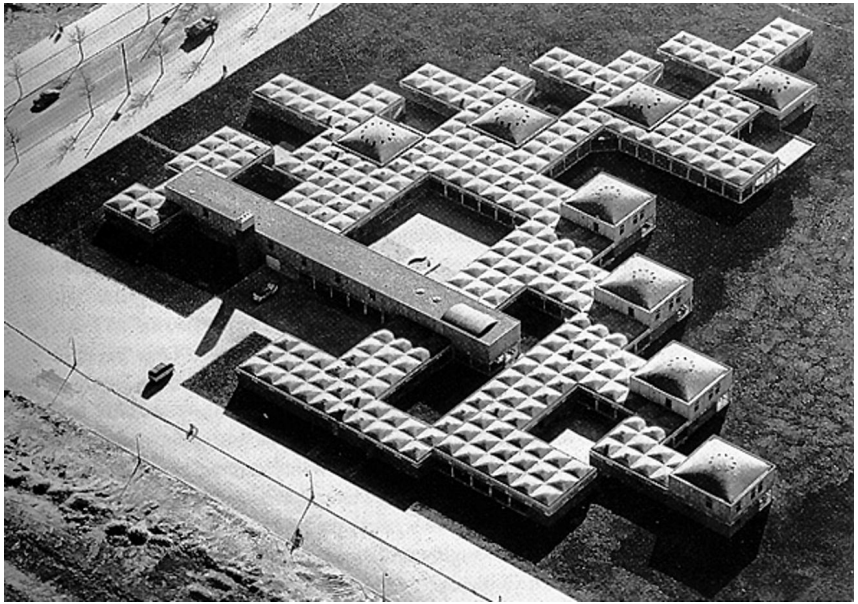
Villa Almerico (Villa Rotunda), from I quattro libri dell'architettura di Andrea Palladio (Book 2, page 19)

Architecture as a system of proportions


Louis Kahn, Exeter Library, 1972, Exterior View | © Xavier de Jauréguiberry

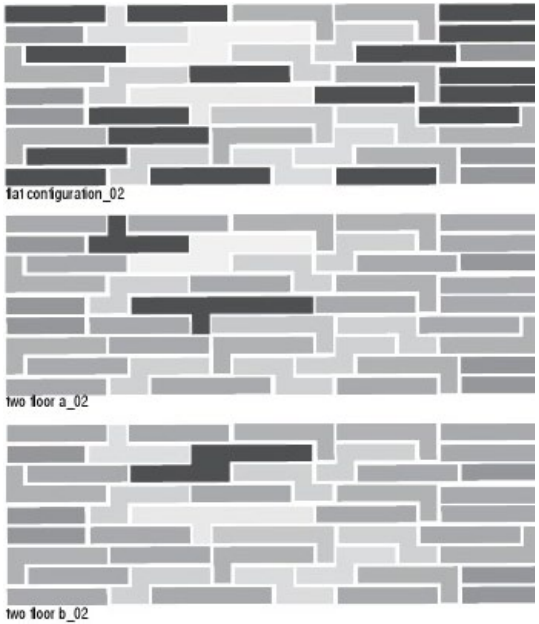
From the whole to relationships between parts





Amsterdam Orphanage, Aldo van Eyck, 1961, Photo Jannes Linders

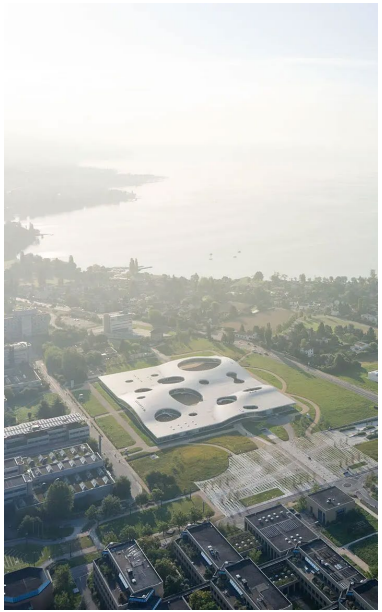
From the whole to relationships between parts





Gifu Kitagata Apartment Building, Kazuyo Sejima, Japan, 1994–2000

From the whole to relationships between parts

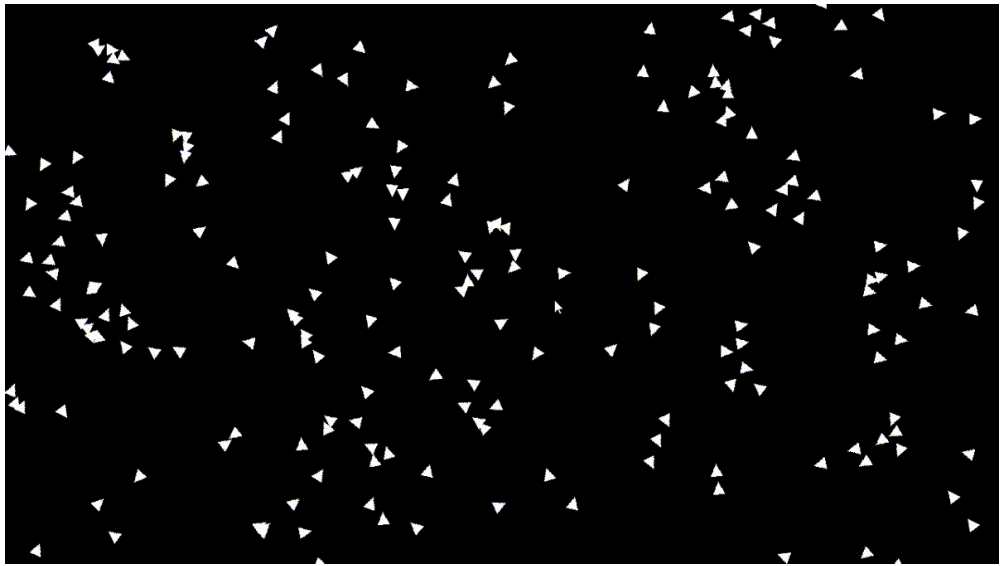






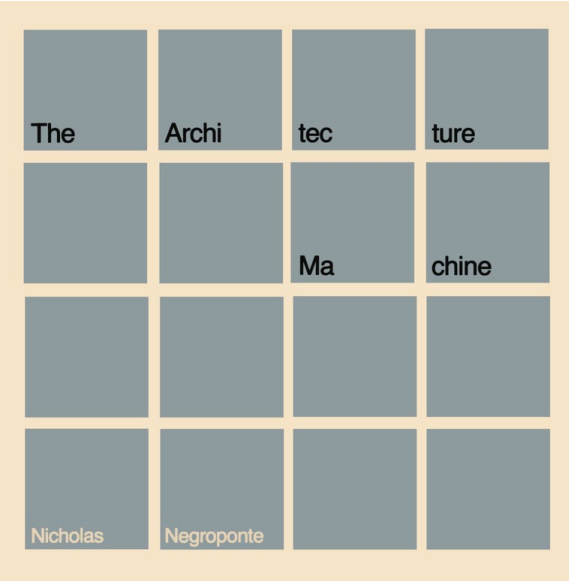
Rolex Learning Center · SANAA, Kazuyo Sejima and Ryue Nishizawa

From the whole to relationships between parts

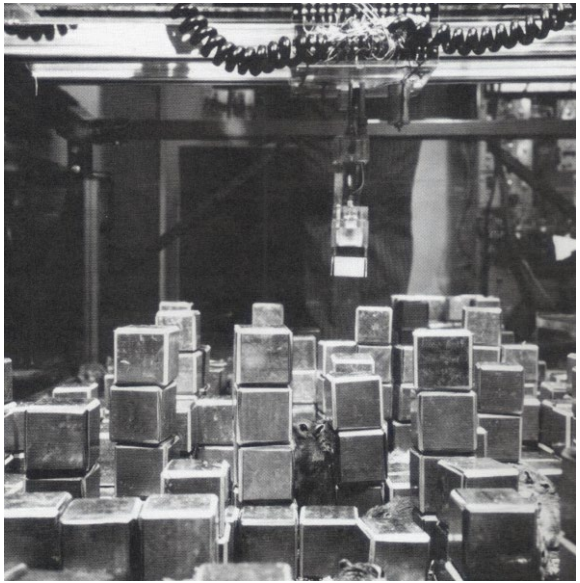


Boids : A flocking algorithm, Craig Reynolds, 1986

From the whole to relationships between parts

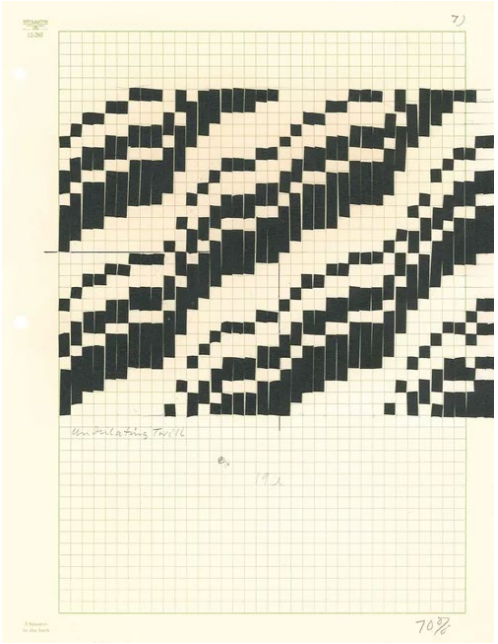


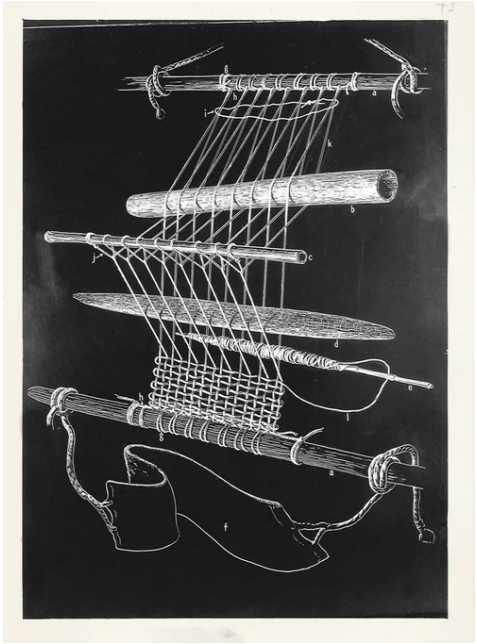
The Architecture Machine: Toward A More Human Environment, 1970, Nicholas Negroponte

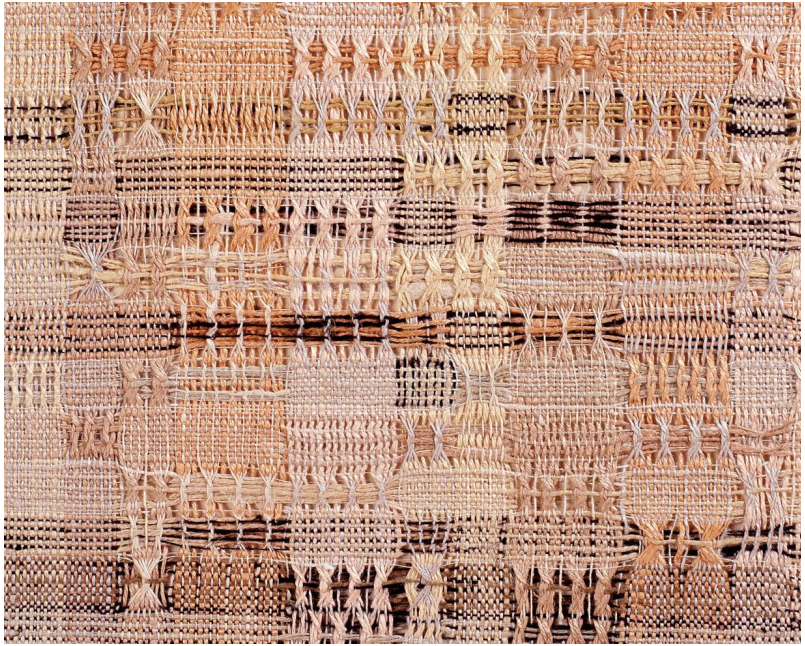


"Seek", 1970 by Nicholas Negroponte with the Architecture Machine Group , M.I.T.

From spatial organisation to processes of making





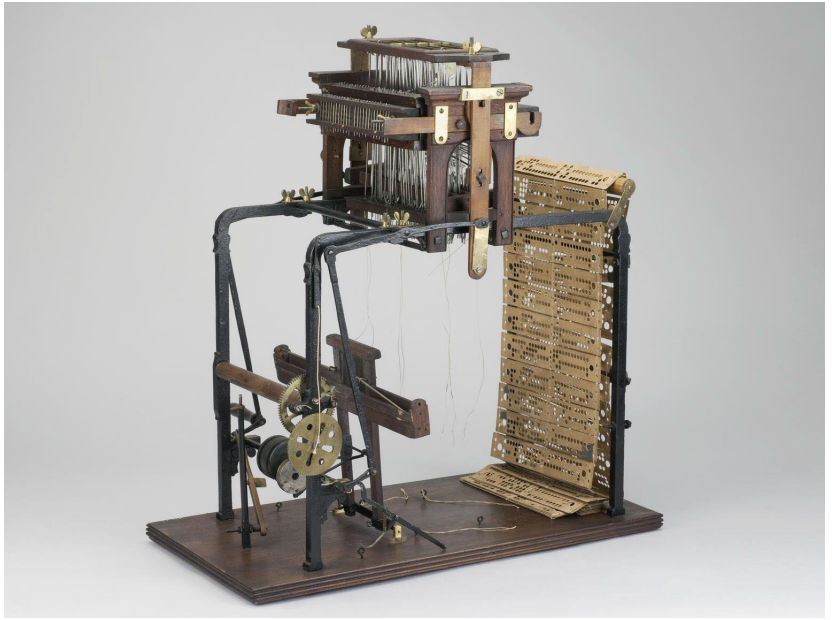


Anni Albers, On Weaving. Originally published in 1965; expanded edition, Princeton University Press, 2017.

From textile production to programmable machinery

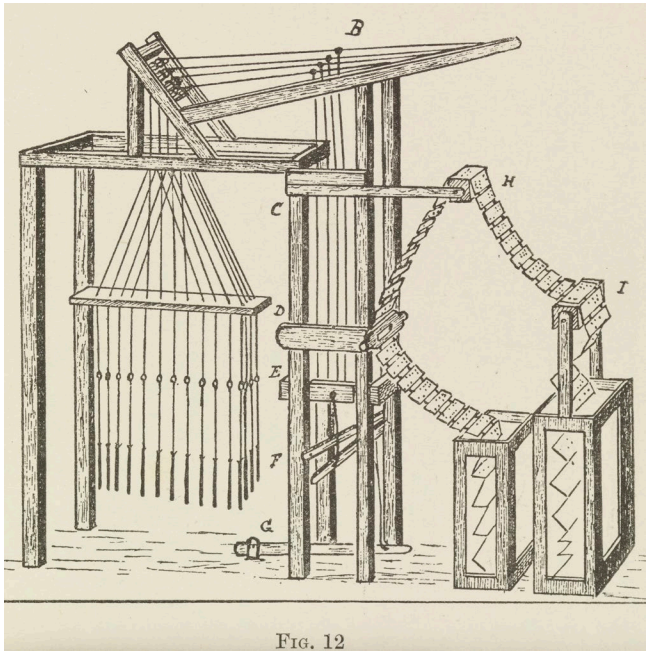






Science and Industry Museum, Programming patterns: the story of the Jacquard loom, 2019.

From textile production to programmable machinery

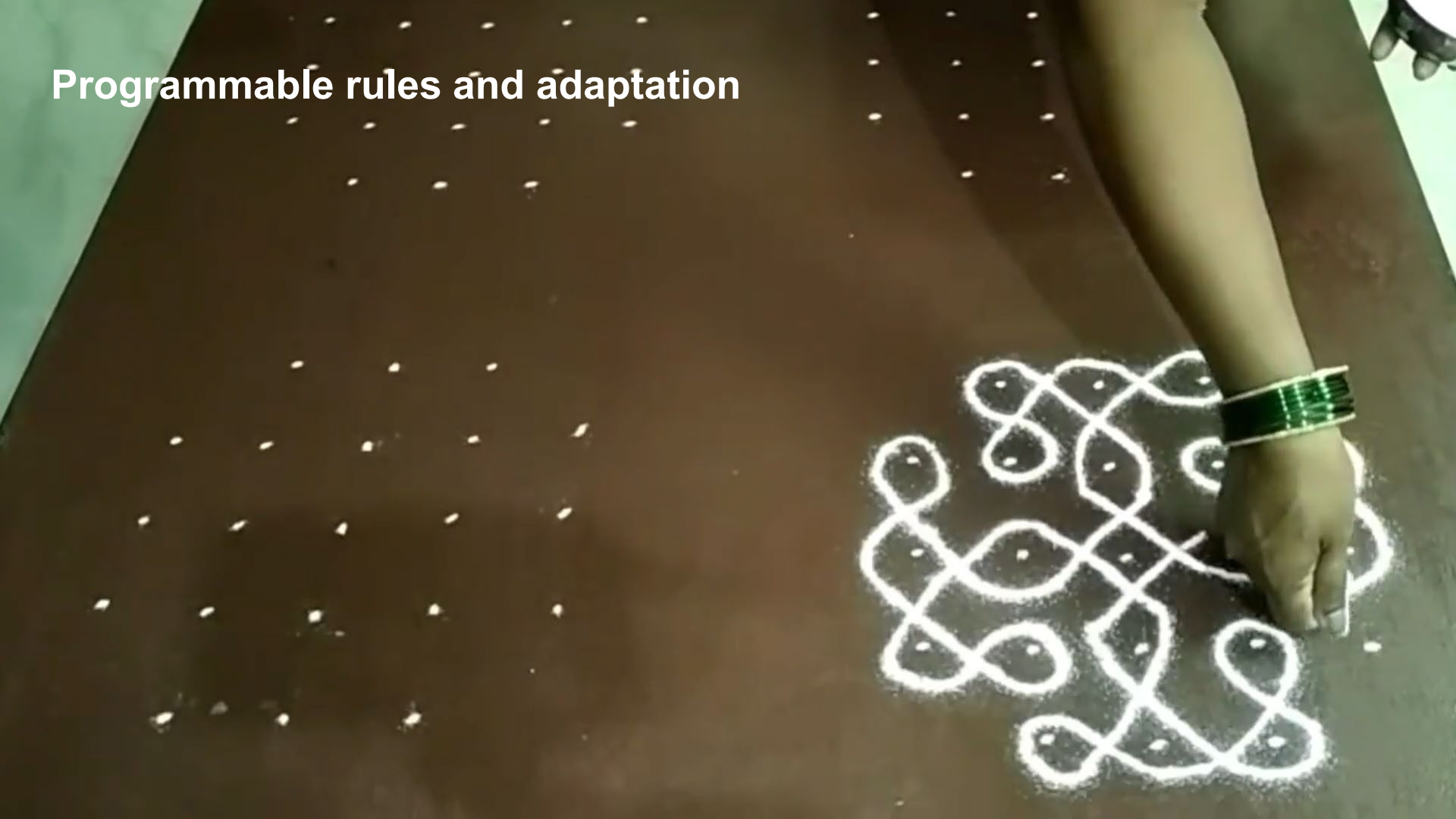






Science and Industry Museum, Programming patterns: the story of the Jacquard loom, 2019 & Emma Terweduwe's Jacquard Woven Textiles.

Programmable rules and adaptation



Algorithmic Design and Digital Fabrication

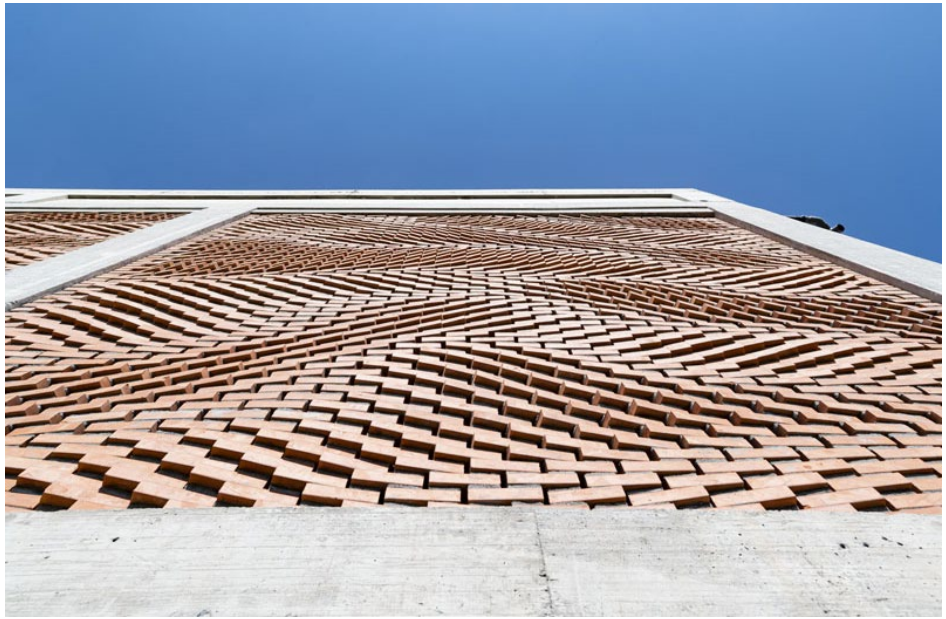
…design becomes both a description of form and instructions for making.





Mesh Mould and In situ Fabricator, ETHZ, 2018.

…design becomes both a description of form and instructions for making.

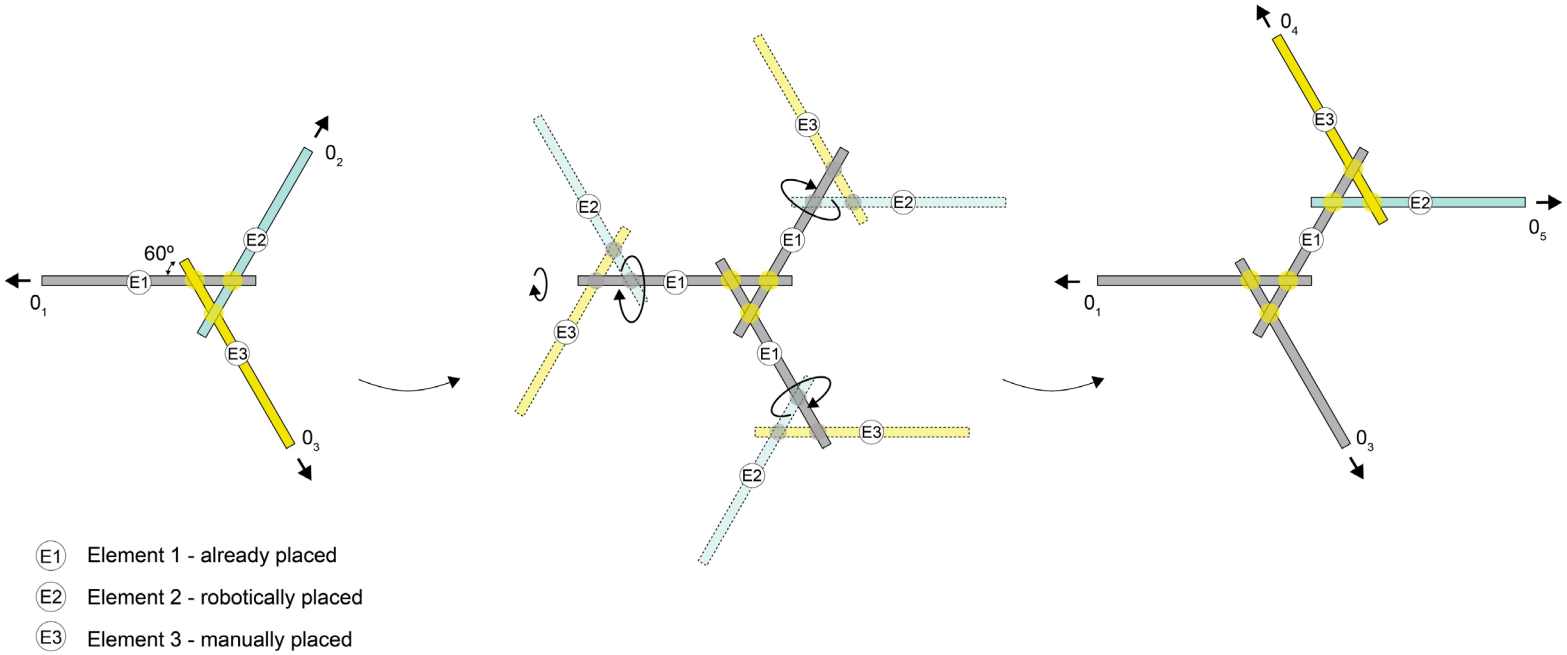




Augmented Bricklaying | Kitrvs Winery, 2019



…we design the structure and its assembly process together

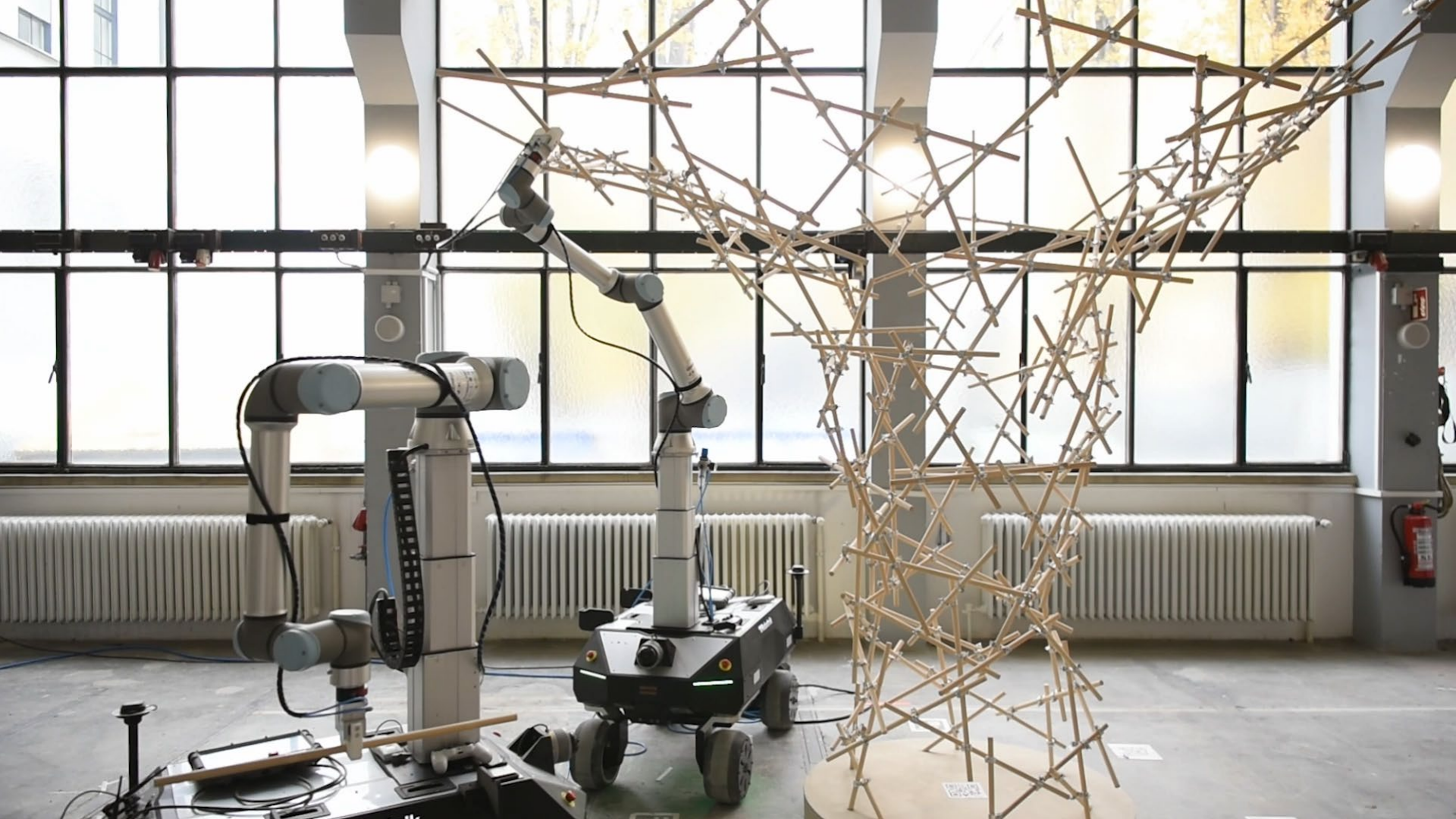


BUILT TOTAL:
0

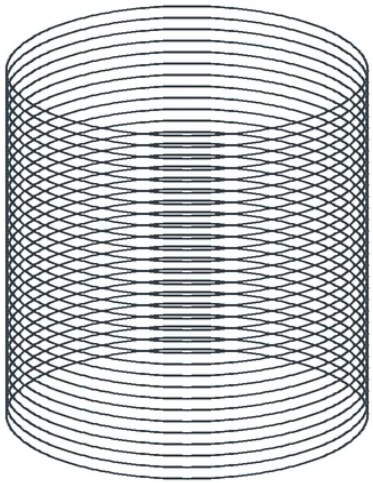
BUILT BY HUMAN:

BUILT BY ROBOT:





…or, we design the structure and its printing process together





Algorithmic Design Approaches

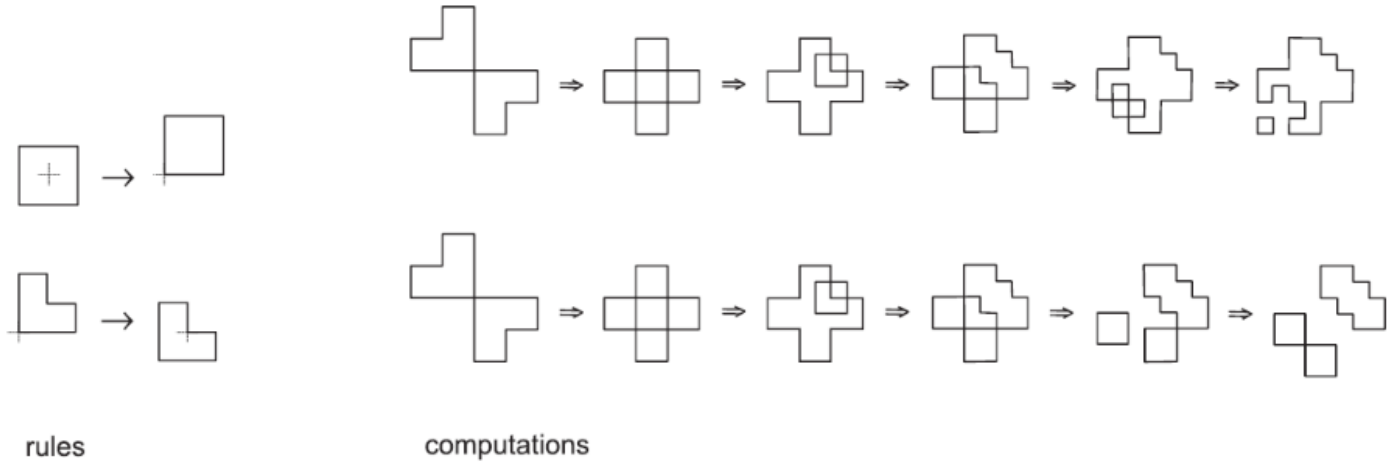
Algorithmic Design Approaches

- **Rule-Based & Procedural Design**
  What transformations are allowed?
- **Growth-Based Design**
  How does a design develop over time?
- **Agent-Based Design**
  How do local actors produce collective organisation?
- **Constraint- & Feasibility-Based Design**
  What configurations are possible?
- **Search-, Optimisation- & Evolution-Based Design**
  Which configurations are preferable?

Rule-Based & Procedural Design Algorithms:

Shape Grammars & Making Grammars

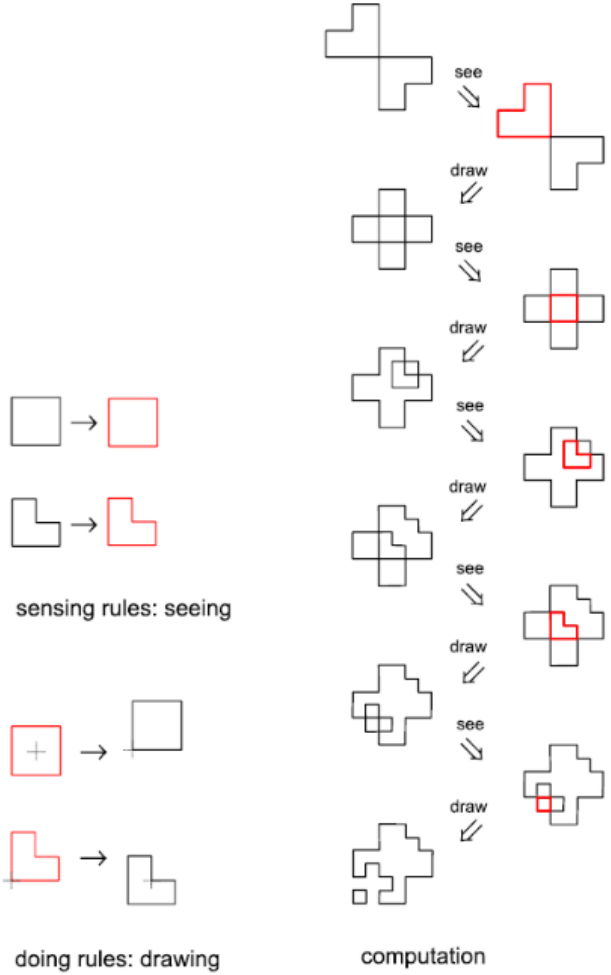
Shape Grammars



Rules of the form A → B

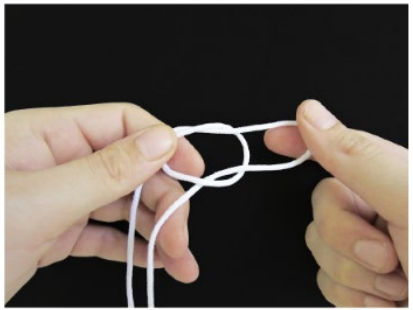
Making grammars: From computing with shapes to computing with things
Terry Knight and George Stiny

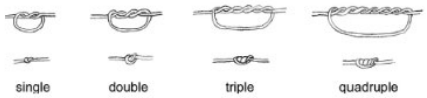
Making Grammars

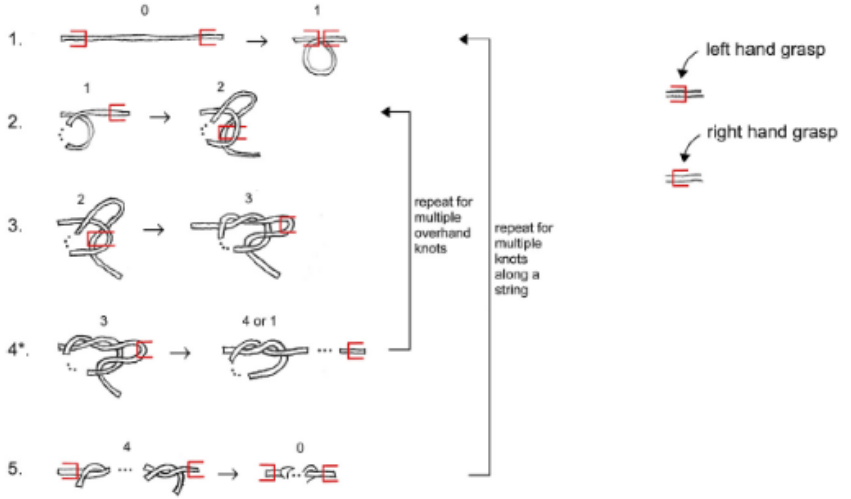


Rules of the form A → B

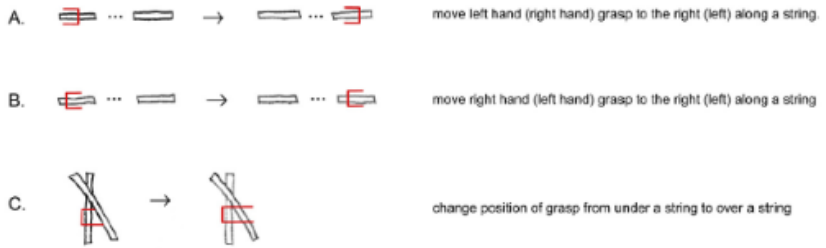
Making Grammars







doing rules: knotting



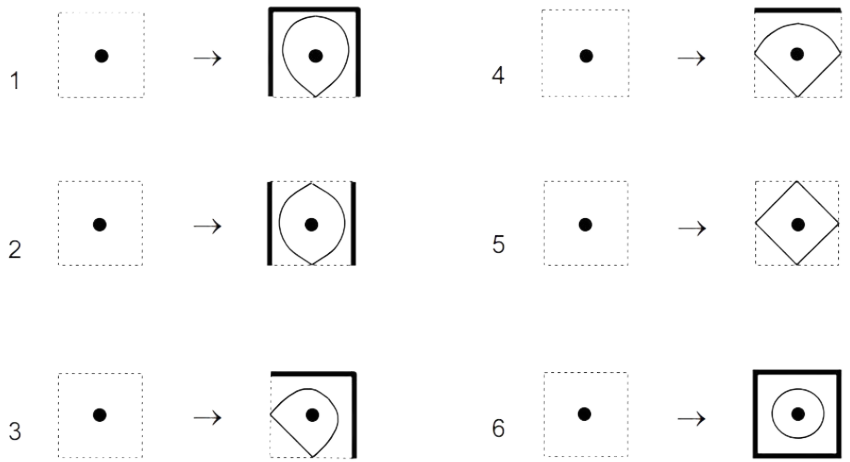
sensing rules: touching, grasping

Exercise 1

Shape Grammar

Simplified modular motif shape grammar:

- **A. Understand the grammar**
- B. Decode the designs
- C. Create your own designs



Computation of a new kolam design with making grammar
Source: Craft, Performance, and Grammars (Knight 2018)

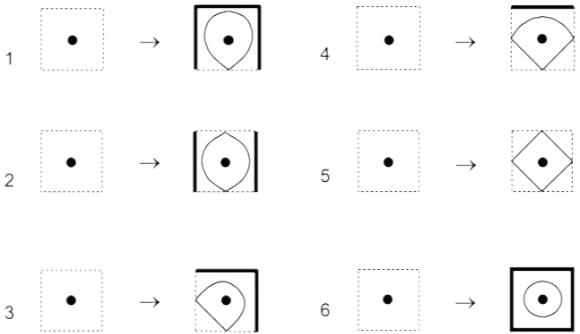
Shape Grammar

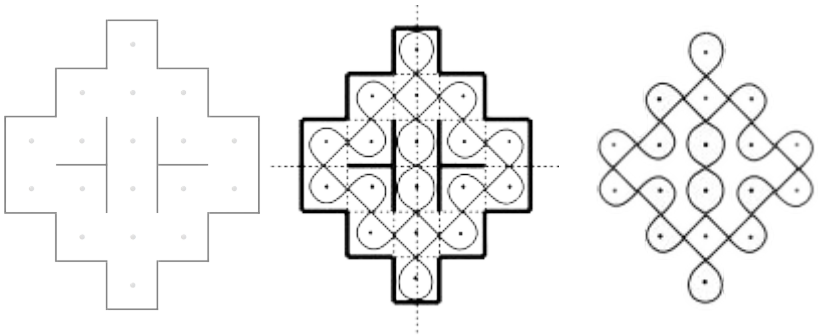
Simplified modular motif shape grammar:

A. Understand the grammar

B. Decode the designs

C. Create your own designs





Computation of a new kolam design with making grammar
Source: Craft, Performance, and Grammars (Knight 2018)

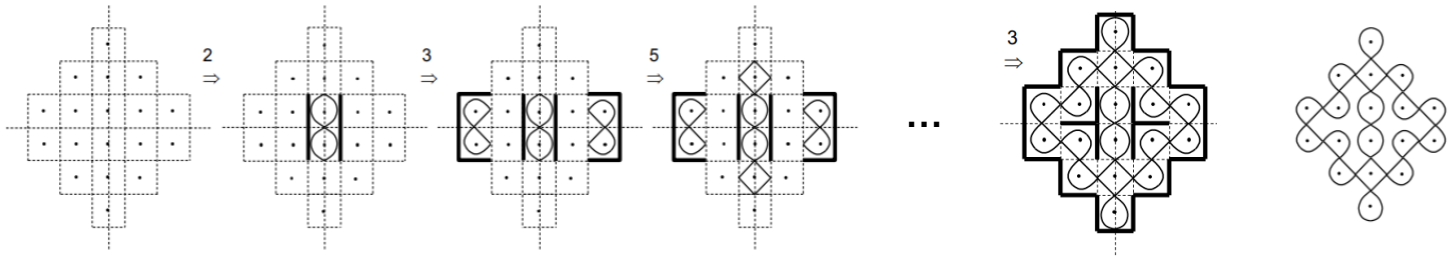
Shape Grammar

Simplified modular motif shape grammar:

A. Understand the grammar

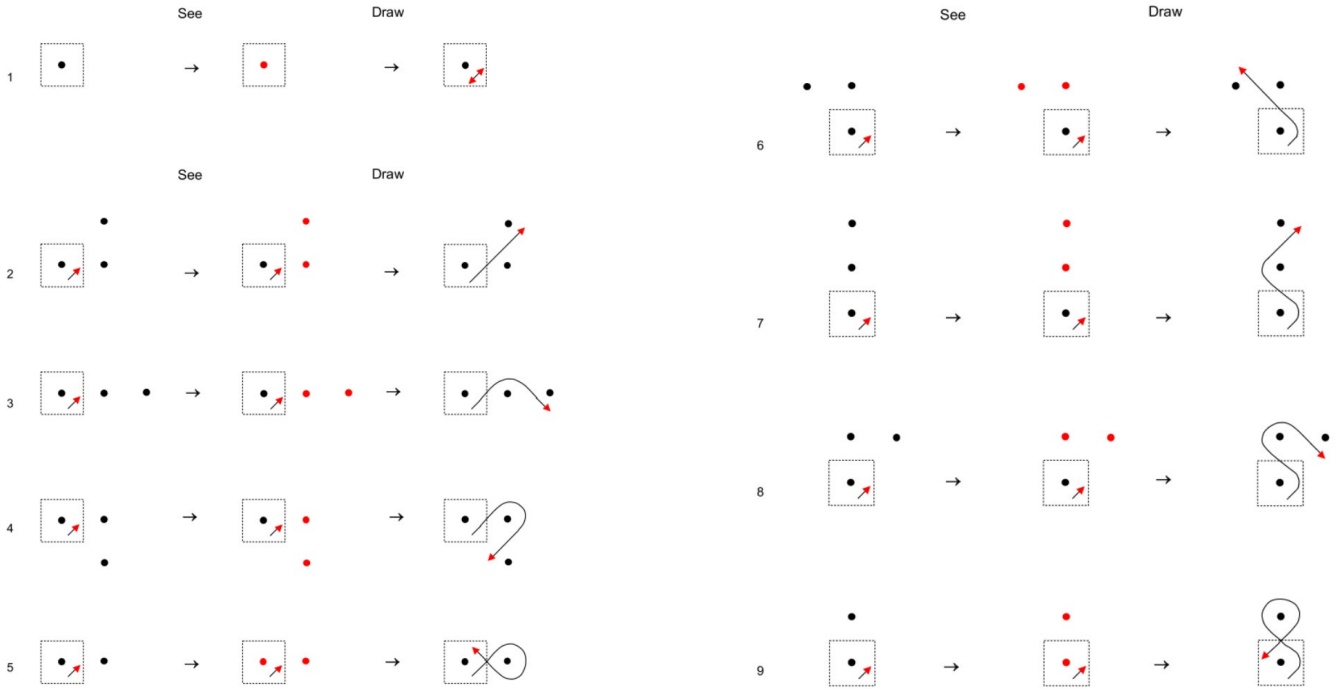
B. Decode the designs

C. Create your own designs



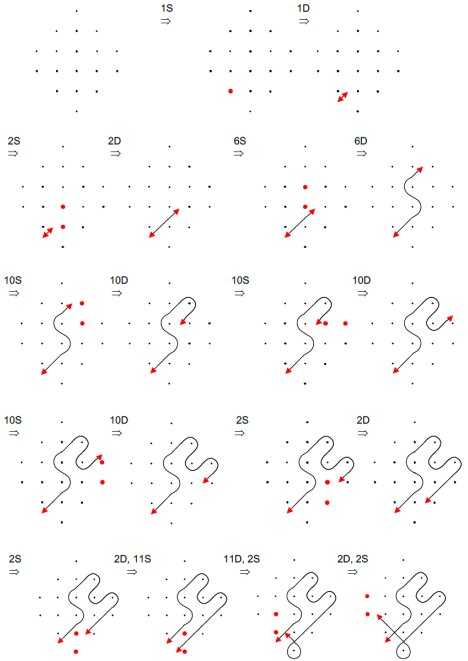
Computation of a new kolam design with making grammar
Source: Craft, Performance, and Grammars (Knight 2018)

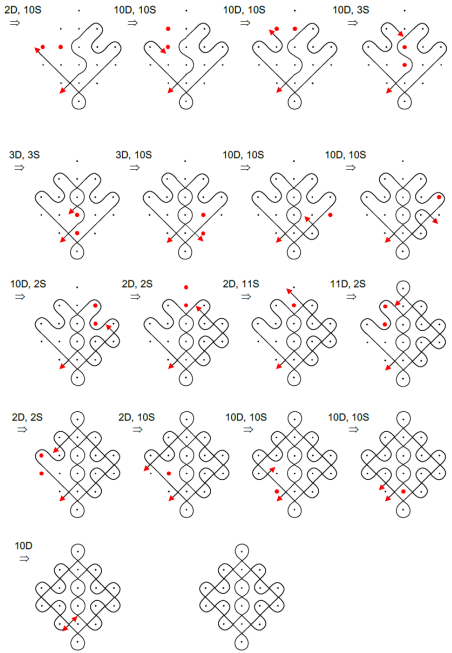
Rule-based design translation



The rules of making grammar
Source: Craft, Performance, and Grammars (Knight 2018)

Rule-based design outputs

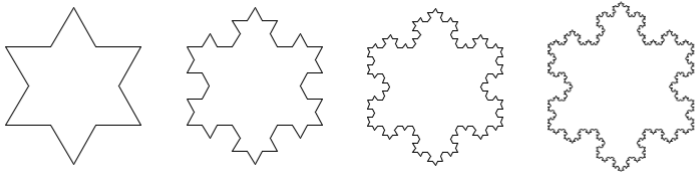




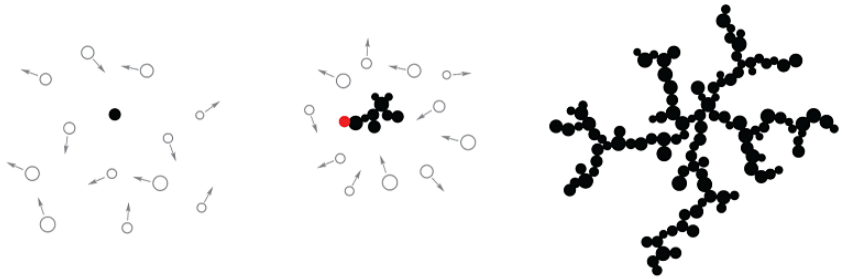
Computation of a new kolam design with making grammar
Source: Craft, Performance, and Grammars (Knight 2018)

Growth-Based Design Algorithms:
Recursive/Dendritic/Differential Growth

Growth-based algorithms



Recursive growth



Sean Collier Memorial at MIT, 2000

Dendrite growth

Recursive Growth (lines)

Learning objectives

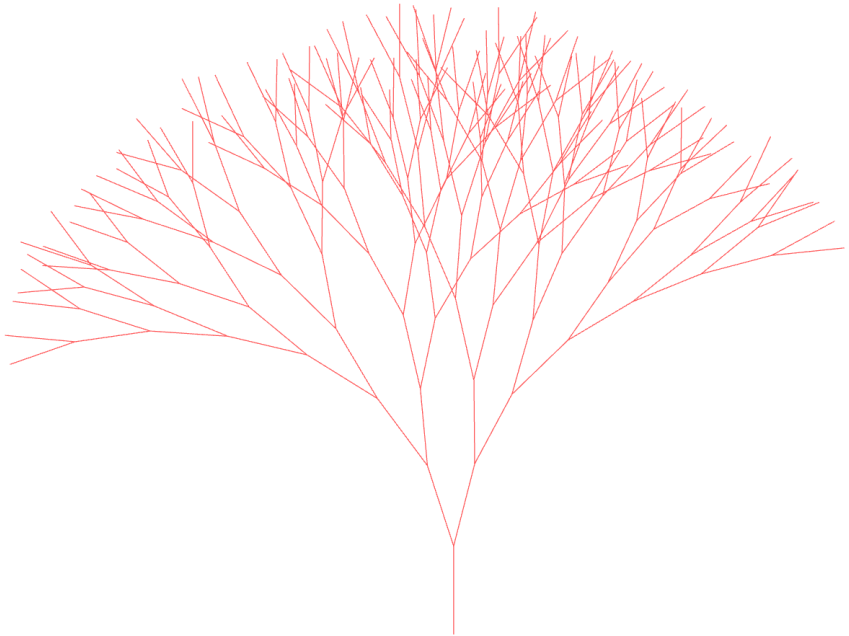
1. Constructing **branching line systems** using **recursion**.
2. **Rotating and scaling vectors** to control directional growth.
3. Applying **stochastic rules to generate variation** in recursive structures.



Recursive Growth (lines)

Learning objectives

1. Constructing **branching line systems** using **recursion**.
2. **Rotating and scaling vectors** to control directional growth.
3. Applying **stochastic rules to generate variation** in recursive structures.



Recursive Growth (lines)

Learning objectives

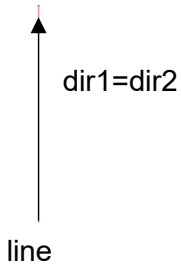
1. Constructing **branching line systems** using **recursion**.
2. **Rotating and scaling vectors** to control directional growth.
3. Applying **stochastic rules to generate variation** in recursive structures.

line

Recursive Growth (lines)

Learning objectives

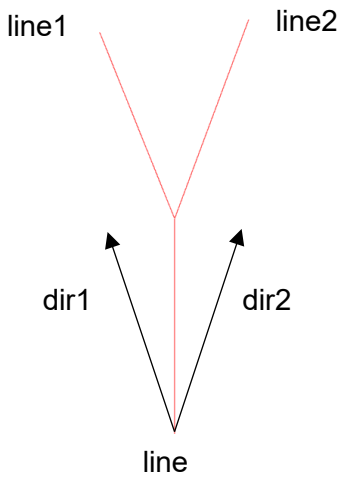
1. Constructing **branching line systems** using **recursion**.
2. **Rotating and scaling vectors** to control directional growth.
3. Applying **stochastic rules to generate variation** in recursive structures.



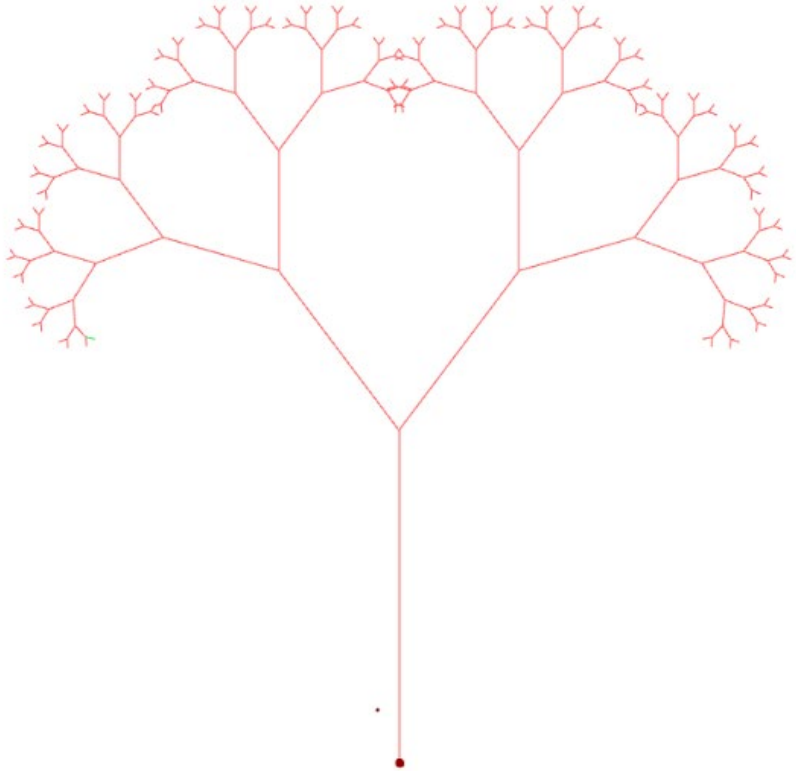
Recursive Growth (lines)

Learning objectives

1. Constructing **branching line systems** using **recursion**.
2. **Rotating and scaling vectors** to control directional growth.
3. Applying **stochastic rules to generate variation** in recursive structures.



Computational Representation

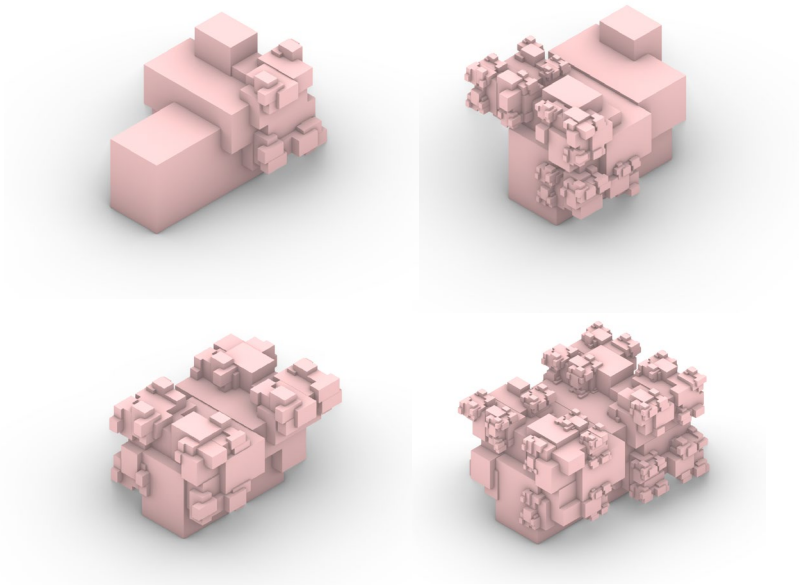


See: cdf1_teaching/exercises/algorithmic-design/**recursive_growth.gh**

Recursive Growth (boxes)

Learning objectives

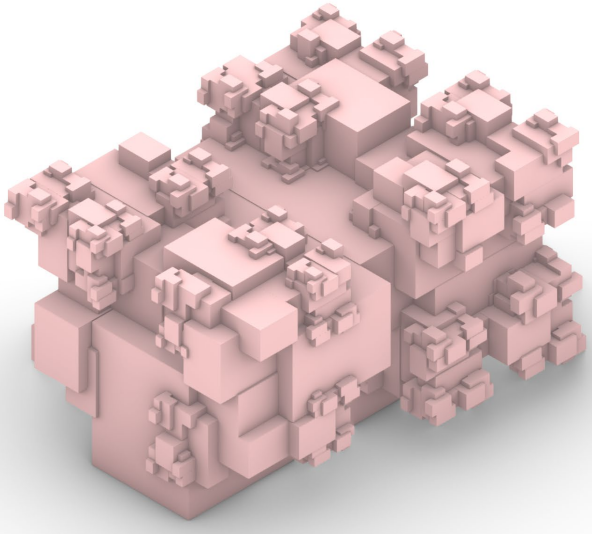
1. Generating **volumetric boxes from recursive line growth**.
2. **Constructing local coordinate frames** using cross products.
3. Creating bounding **volumes aligned to dynamically oriented geometry**.



Recursive Growth (boxes)

Learning objectives

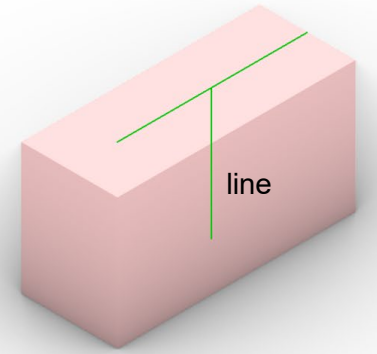
1. Generating **volumetric boxes from recursive line growth**.
2. **Constructing local coordinate frames** using cross products.
3. Creating bounding **volumes aligned to dynamically oriented geometry**.



Recursive Growth (boxes)

Learning objectives

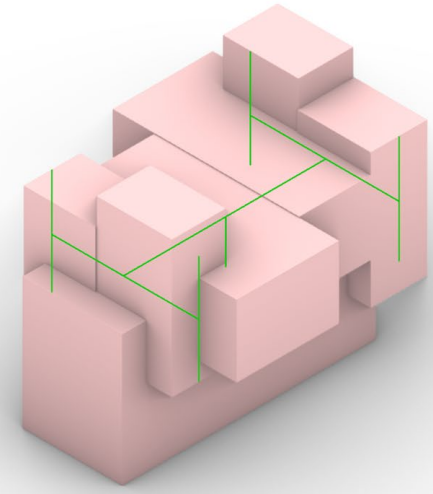
1. Generating **volumetric boxes from recursive line growth**.
2. **Constructing local coordinate frames** using cross products.
3. Creating bounding **volumes aligned to dynamically oriented geometry**.



Recursive Growth (boxes)

Learning objectives

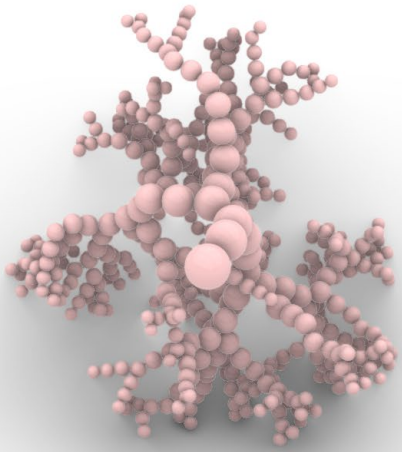
1. Generating **volumetric boxes from recursive line growth**.
2. **Constructing local coordinate frames** using cross products.
3. Creating bounding **volumes aligned to dynamically oriented geometry**.



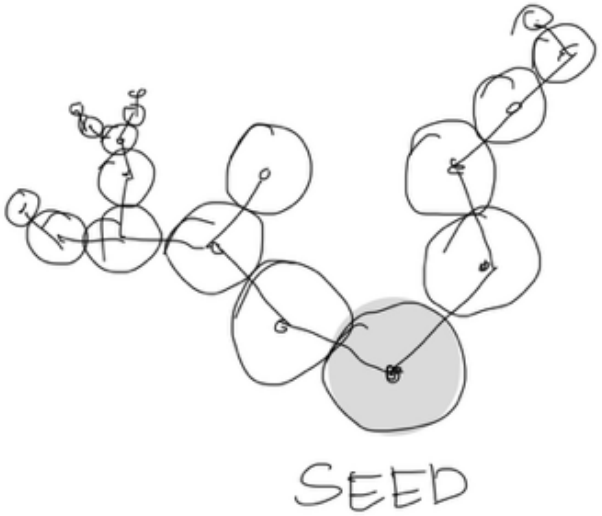
Dendrite Growth

Learning objectives

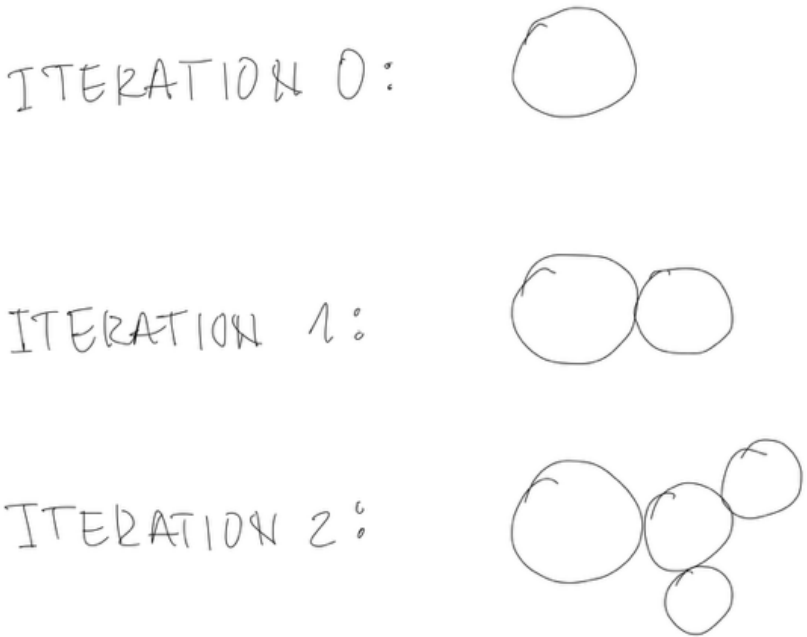
1. **Constructing sphere aggregates** through iterative tip-based growth.
2. Deflecting growth directions in three-dimensional space.
3. Preventing geometric overlap using **spatial collision checks**.



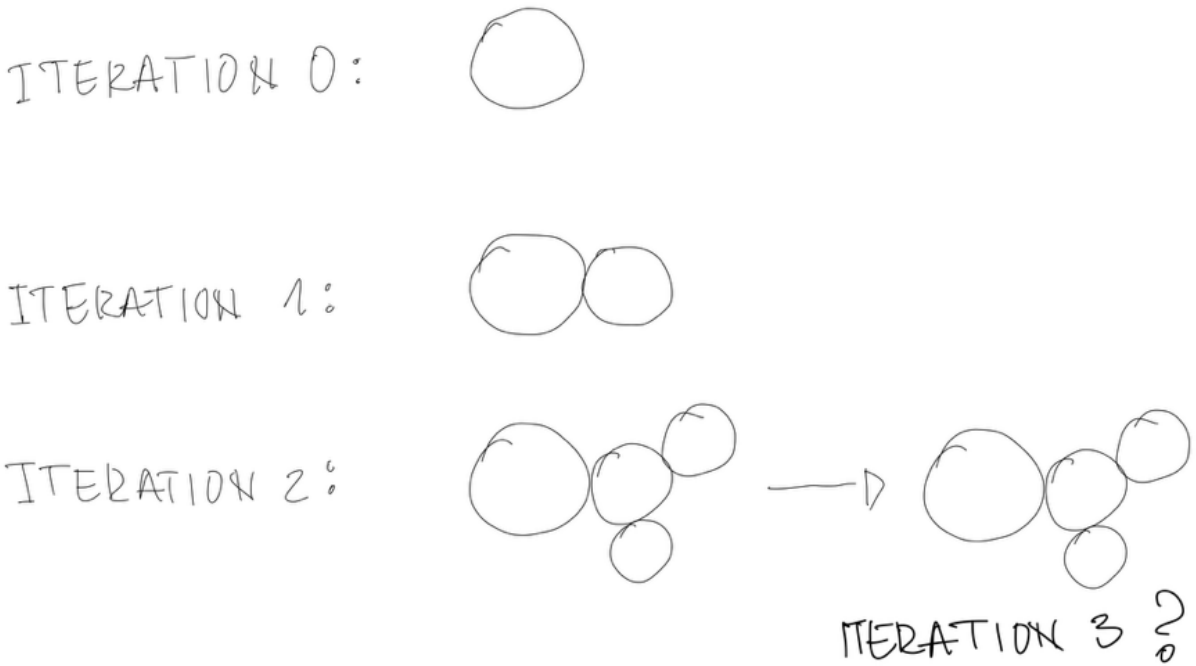
Dendrite Growth



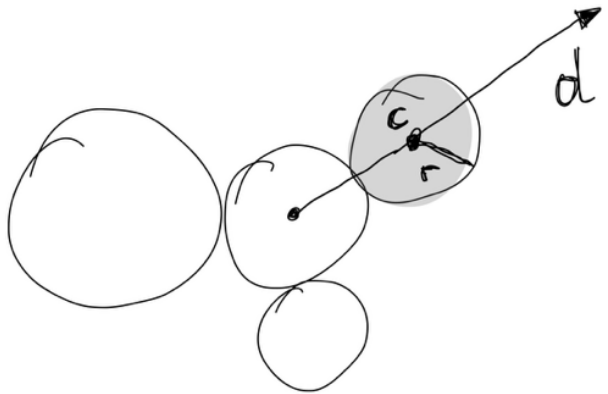
Dendrite Growth



Dendrite Growth

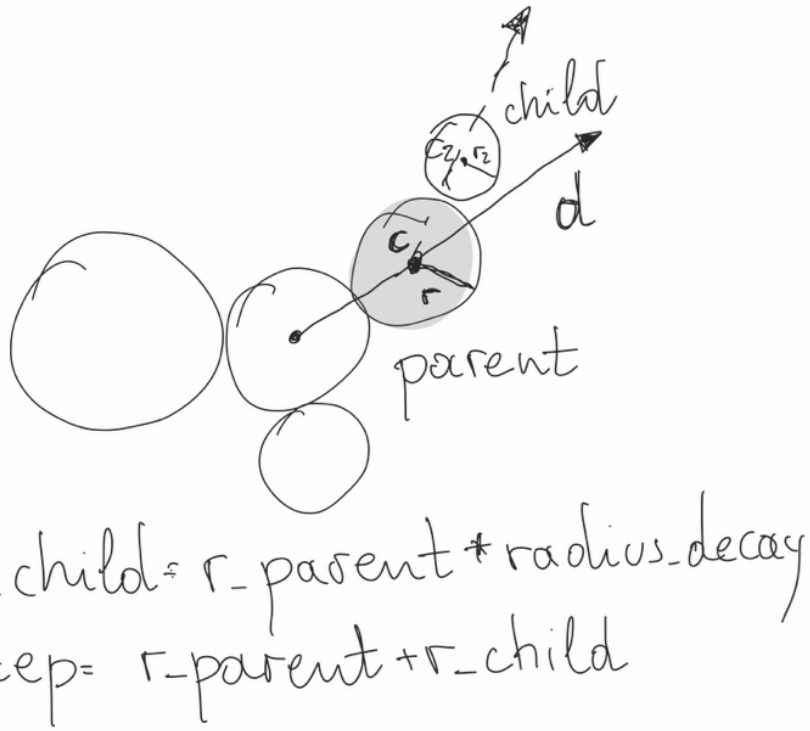


Dendrite Growth

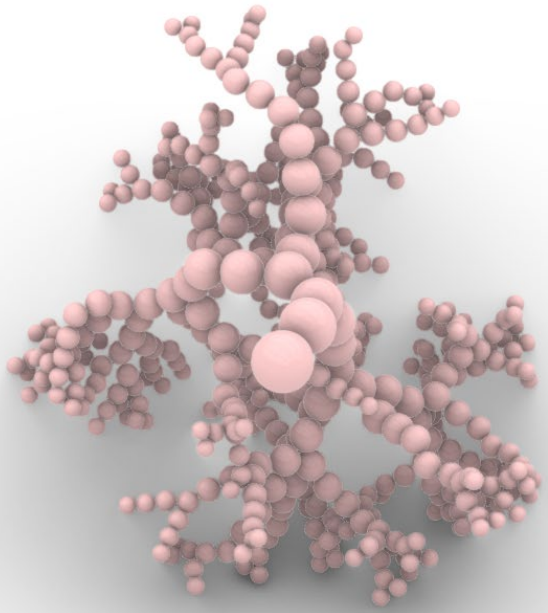


tips = [center_pt, direction, radius]

Dendrite Growth



Computational Representation

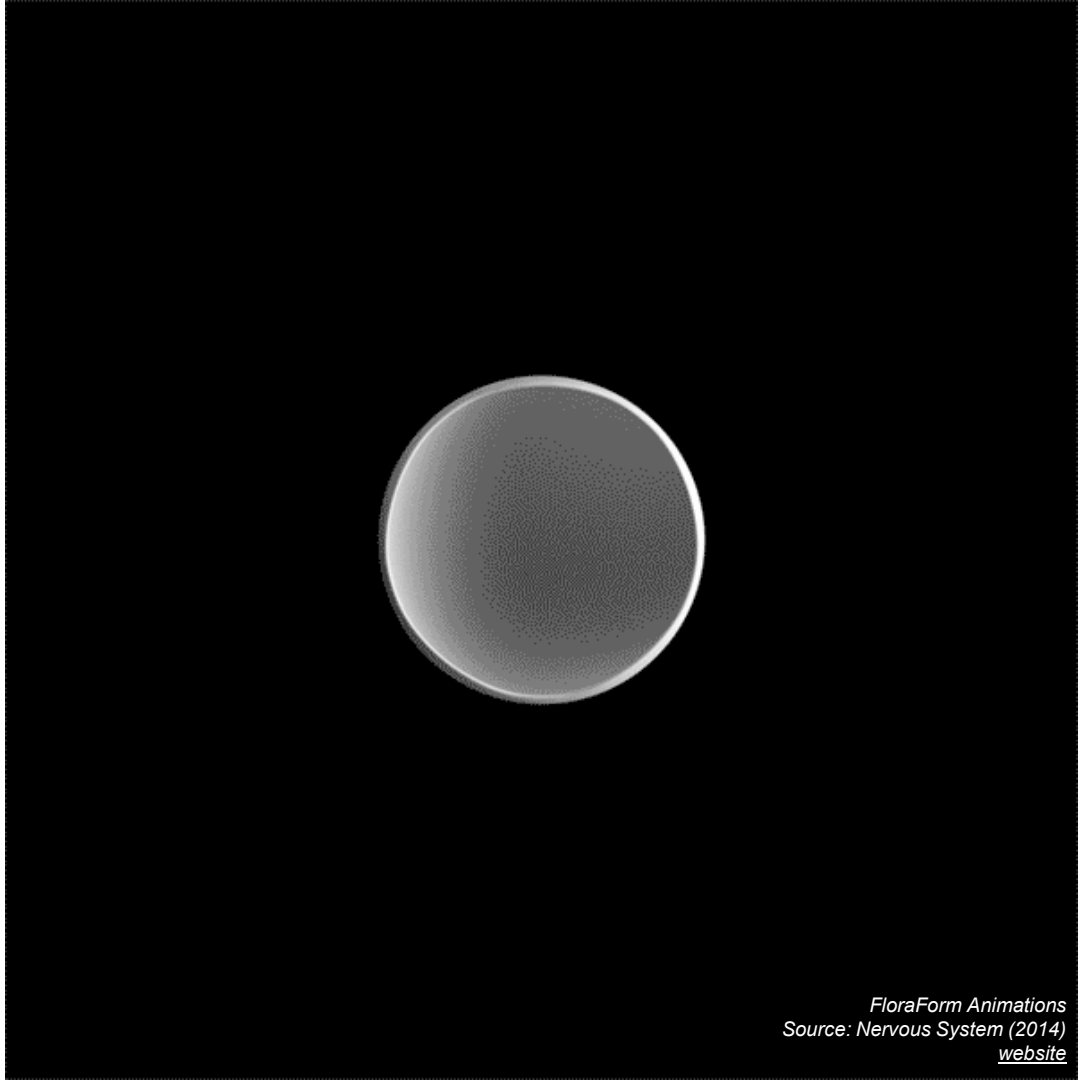


See: cdf1_teaching/exercises/algorithmic-design/**dendrite_growth_spheres.gh**

Differential Growth

FloraForm Animations
Source: Nervous System (2014)
website

Differential Growth



FloraForm Animations
Source: Nervous System (2014)
website

Exercise 2


Crochet Hyperbolic Forms

Example 1: Pseudosphere



```
PROCEDURE CrochetPseudosphere(base_chain_stitch_count=4,
    first_round_stitch_count=12, rounds=5, increase=3)

  // 1. Start and make a base line
  current_layer = 0
  CALL make_a_starting_slipknot() // go to video for instructions

  FOR index = 1 TO base_chain_stitch_count
    CALL make_a_chain_stitch() // go to video for instructions
  END FOR

  // 2. Join to the first stitch to make a ring
  CALL make_a_slipstitch_to_first_chain()  // go to video for instructions

  // 3. Make double crochets into the ring center
  current_stitches_in_round = 0
  FOR i = 1 TO first_round_stitch_count
    CALL double_crochet_into_the_ring_center()
    current_stitches_in_round = current_stitches_in_round + 1
  END FOR
  current_layer = current_layer + 1

  // 4. Hyperbolic expansion in spiral: Make "increase" times double crochet to each
stitch
  FOR r = 1 TO rounds
    FOR s = 1 TO current_stitches_in_round
      FOR k = 1 TO increase
        CALL double_crochet_to_stitch() // go to video for instructions
      END FOR
    END FOR
    current_stitches_in_round = current_stitches_in_round * increase
    current_layer = current_layer + 1
  END FOR

  CALL finish_pattern() // go to video for instructions

END PROCEDURE
```

Design Parameters / Rules

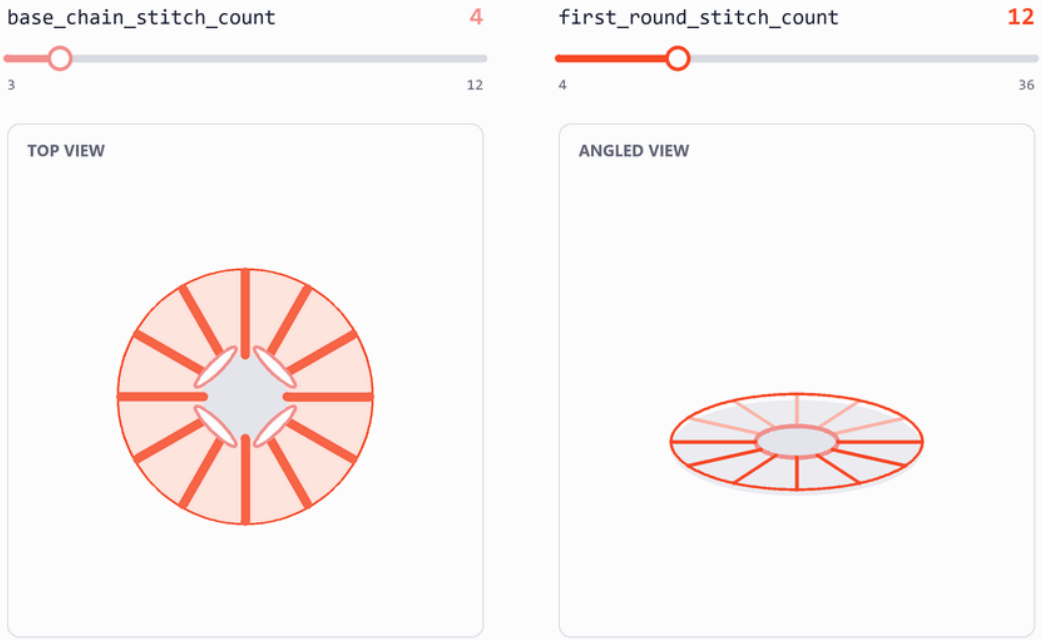
Starting Parameters

- **Base chain stitch count** → size of the centre ring
- **First round stitch count** → starting circumference; scales everything after it

Design Parameters / Rules

Starting Parameters

- Base chain stitch count
- First round stitch count



Design Parameters / Rules

Ruling Parameters

- **Rounds** → how many times the rule repeats; this controls the size
- **Increase** → how many stitches you make into each stitch; this controls the shape
  1 = tube · 2 = ruffled · 3 = very ruffled

How does it change the result?

Starting from 12 stitches in the first round:

Stitches per round = first round × increase^{round},

so the piece grows exponentially.

Increase	Rounds	Stitches in last round	Total stitches
2	3	96	168
2	5	384	744
3	3	324	468
3	5	2,916	4,356

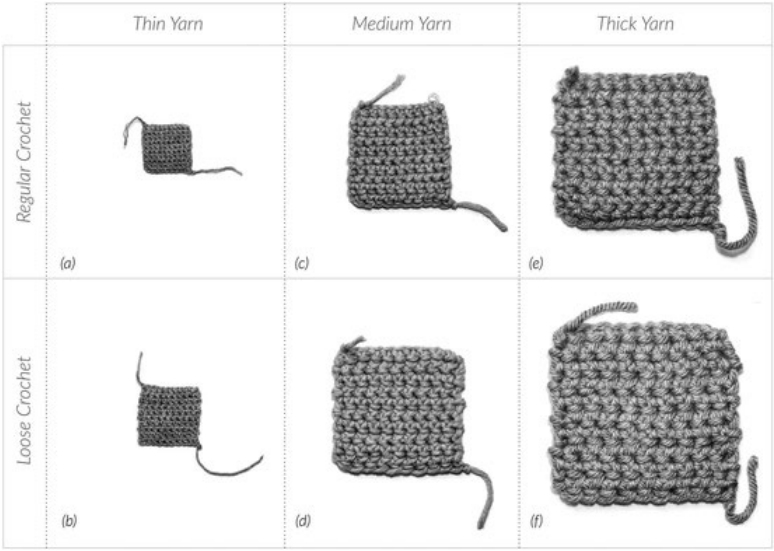
Design Parameters / Rules

Ruling Parameters

- Rounds
- Increase



Design Parameters / Rules



Çapunaman, Ö.B., Bingöl, C.K., Gürsoy, B. (2017). Computing Stitches and Crocheting Geometry

Hidden Parameters *(not in the code)*

- **Stitch width** → set by yarn thickness and hook size
- **Tension** → how tightly you crochet
- **Stitch height** → set by the stitch type: single / double / triple crochet

Computational Representation



See: cdf1_teaching/exercises/algorithmic-design/**differential_growth.gh**

Example 2: Hyperbolic Surface



```
PROCEDURE CrochetHyperbolicSurface(base_chain_stitch_count=16, rows=3, increase=2)

  // 1. Start and make base line
  current_layer = 0
  CALL make_a_starting_slipknot() // go to video for instructions

  current_stitches_in_line = 0
  FOR i = 1 TO base_chain_stitch_count
    CALL make_a_chain_stitch() // go to video for instructions
    current_stitches_in_line = current_stitches_in_line +1
  END FOR

  // 2. Hyperbolic expansion in rows: Make "increase" times double crochet to each stitch
  FOR r = 1 TO rows
    FOR s = 1 TO current_stitches_in_line
      FOR k = 1 TO increase
        CALL double_crochet_in_line()
      END FOR
    END FOR
    current_stitches_in_line = current_stitches_in_line * increase
    current_layer = current_layer + 1

    CALL turn_work()
  END FOR

  CALL finish_pattern()// go to video for instructions

END PROCEDURE
```

Video Instructions

You can find the instructional videos for crocheting hyperbolic forms on **moodle**!

Follow the function names in the pseudocode (e.g. *make_a_starting_slipknot())* and step-by-step build up your crochet piece.



Assignments

Assignment 2.1



Create a computational model of the hyperbolic surface.

Deliverables:

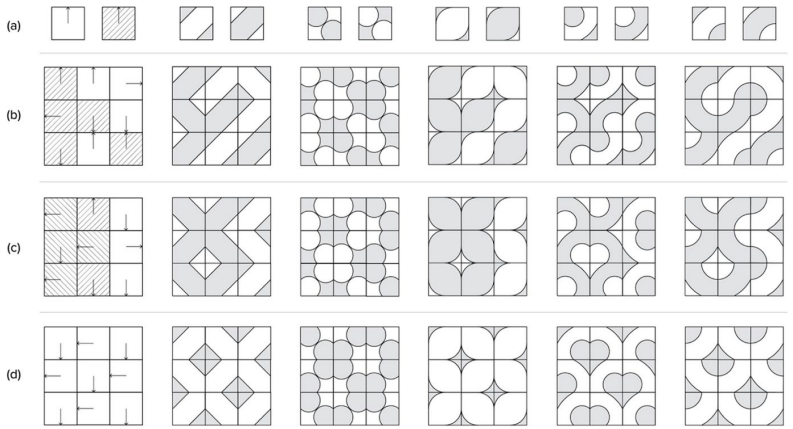
- Rhino/GH Source file with Python Code
- 3 slides of results (Design/Representation Idea, Algorithm, Result (Physical and/or Digital))

Assignment 2.2

Design a floor pattern with an algorithmic approach.

Deliverables:

- Rhino/GH Source file with Python Code
- 3 slides of results (Design Idea, Algorithm, Result)



The Truchet Tile Grammar: A Generative System for Versatile Tile and Pattern Design, Natalie Walter, Heather Ligler & Benay Gürsoy

https://link.springer.com/article/10.1007/s00004-024-00779-8#citeas

Submission Deadlines

16th of November:

- Short Presentation + Feedback session
- Submission of the slides to **Moodle**

23rd of November:

- Final Submission of the slides to Moodle
- **GitHub Submission** of Rhino/GH Source file with Python Code

Enjoy creating!